

Generación de frecuencias seleccionables mediante el microcontrolador ATmega328P

Alexis Aldair Romero Primitivo

Universidad Nacional Autónoma de México, Facultad de Estudios Superiores Cuautitlán

Microcontroladores - Actividad I

27 de septiembre de 2026

Resumen

En el presente trabajo se desarrolla un sistema digital basado en el microcontrolador ATmega328P para generar señales cuadradas con cuatro frecuencias seleccionables: 100 kHz, 500 kHz, 1 MHz y 2 MHz. Dos pines digitales, PD2 y PD3, se configuran como entradas y determinan la frecuencia de operación, mientras que PB5 se utiliza como salida de la señal. El programa fue escrito en lenguaje ensamblador AVR y ensamblado en Microchip Studio. La generación temporal se realizó a partir del número de ciclos de instrucción ejecutados por el procesador a 16 MHz, ajustando el intervalo entre cambios de estado de PB5. El funcionamiento fue comprobado en SimulIDE mediante interruptores digitales y un frecuencímetro, verificándose la selección de las cuatro frecuencias. Finalmente, el archivo hexadecimal generado fue programado y verificado en la memoria Flash de una tarjeta Arduino Nano con microcontrolador ATmega328P, con lo cual se obtuvo una implementación preparada para su comprobación física.

1. Introducción

Los microcontroladores integran en un solo circuito una unidad de procesamiento, memoria y periféricos de entrada y salida, lo que permite implementar sistemas embebidos con una cantidad reducida de componentes externos. El ATmega328P pertenece a la familia AVR de 8 bits y cuenta con memoria de programa, memoria de datos, puertos digitales y periféricos temporizadores, de comunicación y conversión analógico-digital [1].

En esta actividad se empleó el ATmega328P para implementar un generador digital de frecuencias controlado mediante dos entradas binarias. La combinación lógica de PD3 y PD2 selecciona una señal de 100 kHz, 500 kHz, 1 MHz o 2 MHz, mientras que la señal cuadrada se obtiene en PB5, correspondiente al pin digital D13 de la tarjeta Arduino Nano utilizada. La tabla de selección establecida para el sistema es 00 para 100 kHz, 01 para 500 kHz, 10 para 1 MHz y 11 para 2 MHz.

El programa se desarrolló en ensamblador AVR con Microchip Studio, entorno que permite crear, ensamblar y depurar proyectos para dispositivos AVR [4]. La simulación se efectuó con SimulIDE, herramienta que permite ejecutar firmware de microcontroladores y observar señales mediante instrumentos virtuales como osciloscopio y frecuencímetro [5]. La tarjeta Arduino Nano trabaja a 5 V y utiliza un reloj de 16 MHz, por lo que cada ciclo de reloj tiene una duración de 62.5 ns [3]. Esta relación temporal se aprovechó para calcular el número de ciclos requerido entre cambios de estado de PB5.

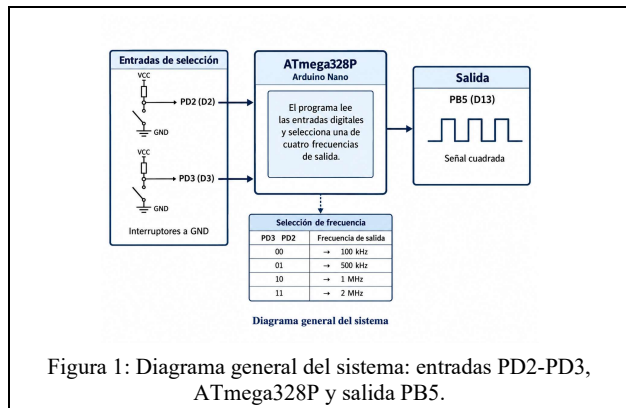


Figura 1: Diagrama general del sistema: entradas PD2-PD3, ATmega328P y salida PB5.

2. Funcionamiento del sistema

El sistema está formado por tres bloques principales: las entradas de selección, el microcontrolador y la salida de frecuencia. Las entradas PD2 y PD3 reciben niveles lógicos mediante interruptores conectados a tierra. Debido a que se habilitan las resistencias pull-up internas, un interruptor abierto se interpreta como nivel lógico 1 y un interruptor cerrado como nivel lógico 0. El programa lee ambos bits, determina la combinación seleccionada y ejecuta la rutina temporal correspondiente. PB5 cambia periódicamente entre niveles bajo y alto para producir una señal cuadrada.

2.1. Microprocesador AVR ATmega328P

El ATmega328P es un microcontrolador AVR de 8 bits basado en una arquitectura RISC mejorada. El núcleo utiliza una arquitectura Harvard, por lo que la memoria de programa y la memoria de datos se encuentran separadas. Dispone de 32 registros de propósito general de 8 bits conectados directamente a la unidad aritmético-lógica y puede alcanzar un rendimiento de hasta 16 MIPS cuando opera a 16 MHz [1].

En el programa se emplean principalmente R16 y R17. R16 almacena temporalmente el valor leído desde PIND; posteriormente se aplica la máscara 00001100b para conservar únicamente los estados de PD3 y PD2. R17 se utiliza como contador en la rutina correspondiente a 100 kHz. Las instrucciones CPI y DEC modifican las banderas del registro de estado SREG, en particular la bandera Z, que es evaluada por BREQ y BRNE para realizar saltos condicionales [2].

Registro(s)	Función dentro del programa
R16	Almacena y enmascara la lectura de PD2 y PD3.
R17	Contador empleado para el retardo de 100 kHz.
DDRB / DDRD	Define PB5 como salida y PD2-PD3 como entradas.
PORTB / PORTD	Inicializa PB5 y habilita los pull-up internos.
PINB / PIND	Permite conmutar PB5 y leer el puerto D.
SREG	Sus banderas son evaluadas por los saltos condicionales.

La instrucción SBI permite establecer un bit individual de un registro de E/S y CBI permite borrarlo, ambas sin modificar los

demás bits del registro. También se utiliza la característica del AVR por la cual escribir un uno lógico en un bit de PINB conmuta el bit correspondiente de PORTB; de esta forma SBI PINB, PINB5 invierte directamente el estado de PB5 [1], [2].

2.2. Memorias

El ATmega328P integra tres tipos principales de memoria: 32 kB de memoria Flash de programa, 2 kB de SRAM y 1 kB de EEPROM [1]. Cada una cumple una función diferente dentro del sistema.

La memoria Flash es no volátil y almacena las instrucciones que ejecuta el microcontrolador. El archivo.hex generado al ensamblar el proyecto se graba en esta memoria. Al retirar la alimentación, el contenido permanece almacenado y vuelve a ejecutarse después de un reinicio. En una tarjeta Arduino Nano típica una parte de la Flash puede estar reservada al bootloader; la documentación de la placa oficial indica 32 kB de Flash, de los cuales 2 kB se destinan al bootloader [3].

La SRAM es una memoria volátil destinada a datos temporales durante la ejecución, como variables, pila y almacenamiento de trabajo. En este programa la mayor parte del procesamiento se realiza directamente con registros del núcleo, por lo que el uso explícito de SRAM es reducido. La EEPROM también es no volátil y permite conservar datos aun sin alimentación; sin embargo, no fue necesario utilizarla porque la actividad no requiere almacenar configuraciones o parámetros entre reinicios.

Memoria	Capacidad	Uso en la actividad
Flash	32 kB	Almacena el programa ensamblado.
SRAM	2 kB	Disponible para datos temporales y pila.
EEPROM	1 kB	No utilizada en esta implementación.

2.3. Interfaz para periféricos

El ATmega328P dispone de 23 líneas de entrada/salida de propósito general [1]. Para esta actividad se utilizan los puertos B y D. El bit PB5 se configura como salida mediante DDRB y corresponde al pin D13 del Arduino Nano. PD2 y PD3 se configuran como entradas mediante DDRD y corresponden a D2 y D3. El estado lógico de las entradas se obtiene leyendo PIND.

Las resistencias pull-up internas de PD2 y PD3 se habilitan escribiendo un uno en los bits correspondientes de PORTD mientras dichos pines permanecen configurados como entradas. De esta manera, los interruptores pueden conectarse directamente entre D2 o D3 y GND sin necesidad de resistencias externas de polarización. Los puertos AVR permiten operaciones de lectura-modificación-escritura por bit, característica aprovechada mediante SBI, CBI y SBIS [1].

La tarjeta Arduino Nano utilizada opera con lógica de 5 V y reloj de 16 MHz [3]. Para el ATmega328P, con VCC entre 2.7 V y 5.5 V, un nivel bajo en una entrada digital se garantiza hasta 0.3VCC y un nivel alto desde 0.6VCC. La corriente absoluta máxima indicada por el fabricante del microcontrolador es 40 mA por pin; no obstante, la especificación de la tarjeta Nano indica 20 mA por pin de E/S como valor de operación, por lo que no debe diseñarse una carga que exceda estos límites [1], [3].

Pin AVR	Pin Nano	Configuración	Función
PD2	D2	Entrada + pull-up	Bit menos significativo de selección.
PD3	D3	Entrada + pull-up	Bit más significativo de selección.
PB5	D13	Salida	Generación de la señal cuadrada.
GND	GND	Referencia	Tierra común de interruptores e instrumento.

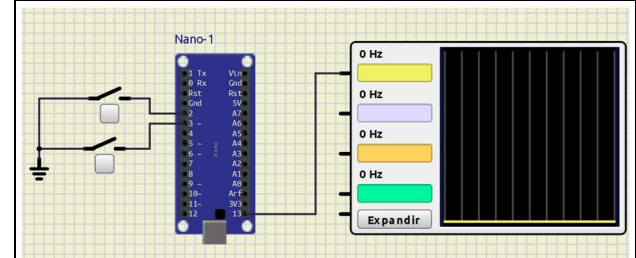


Figura 2: Circuito implementado en SimulIDE con dos interruptores de selección y medición en D13.

2.4. Generación temporal de las frecuencias

La generación de las señales se realiza por software a partir del tiempo de ejecución de las instrucciones. Con una frecuencia de reloj de 16 MHz, el periodo de un ciclo del procesador es $T_{CPU} = 1/16 \text{ MHz} = 62.5 \text{ ns}$. Como PB5 debe cambiar dos veces para completar un periodo de la señal cuadrada, la frecuencia de salida puede expresarse como $f_{out} = F_{CPU}/(2N)$, donde N representa el número de ciclos transcurridos entre dos conmutaciones consecutivas de PB5.

Frecuencia	Periodo	Medio periodo	Ciclos por conmutación
100 kHz	10 us	5 us	80
500 kHz	2 us	1 us	16
1 MHz	1 us	0.5 us	8
2 MHz	0.5 us	0.25 us	4

Las rutinas de 100 kHz, 500 kHz y 1 MHz combinan la conmutación de PB5 con la lectura de PIND y retardos ajustados mediante DEC, BRNE y NOP. Para 2 MHz solo se dispone de cuatro ciclos entre conmutaciones, por lo que se emplea SBIS para comprobar directamente los bits PD2 y PD3 sin exceder el presupuesto temporal. Mientras la selección se mantiene en 11, la secuencia conserva cuatro ciclos entre cambios de PB5; si alguno de los bits cambia a cero, el flujo regresa a MAIN y se selecciona la nueva rutina.

3. Algoritmo del programa

El algoritmo comienza configurando PB5 como salida y PD2-PD3 como entradas con resistencias pull-up. A continuación, se lee el puerto D, se aplica una máscara para conservar únicamente los dos bits de selección y se compara el resultado con las cuatro combinaciones posibles. Cada rutina genera la frecuencia requerida y comprueba periódicamente si la selección continúa siendo válida. Si cambia cualquiera de los bits, el programa vuelve al bloque principal de selección.

Entrada PD3:PD2	Frecuencia de salida
00	100 kHz
01	500 kHz
10	1 MHz
11	2 MHz

Algoritmo 1: Selección y generación de frecuencia

Datos: estados lógicos de PD3 y PD2.

Resultado: señal cuadrada en PB5.

```

1 inicio
2 configurar PB5 como salida
3 configurar PD2 y PD3 como entradas
4 habilitar pull-up en PD2 y PD3
5 repetir
6 leer PIND
7 conservar únicamente PD3 y PD2
8 si PD3:PD2 = 00 entonces generar 100 kHz
9 si PD3:PD2 = 01 entonces generar 500 kHz
10 si PD3:PD2 = 10 entonces generar 1 MHz
11 si PD3:PD2 = 11 entonces generar 2 MHz
12 verificar si cambió la selección
13 hasta que finalice la ejecución
14 fin

```

En la implementación, las comparaciones se realizan con CPI y BREQ. ANDI R16, 00001100b elimina los bits del puerto D que no intervienen en la selección. La conmutación de la salida se efectúa mediante SBI PINB, PINB5. Para 100 kHz se utiliza R17 como contador de retardo, mientras que las frecuencias mayores requieren menos instrucciones entre conmutaciones.

4. Análisis de resultados

La simulación permitió verificar inicialmente cada frecuencia de manera independiente y posteriormente el funcionamiento del selector completo. A 16 MHz se comprobó que la temporización por ciclos de instrucción produce los valores nominales requeridos cuando el número de ciclos entre conmutaciones se ajusta de acuerdo con el medio periodo de cada señal.

Durante la calibración de 100 kHz se obtuvo inicialmente un valor aproximado de 98.77 kHz. El análisis del número de ciclos mostró que la rutina utilizaba 81 ciclos entre conmutaciones en lugar de los 80 requeridos. Al eliminar un ciclo de retardo se corrigió la salida a 100 kHz. Esta prueba mostró la importancia de considerar el tiempo de ejecución de cada instrucción, especialmente a frecuencias elevadas.

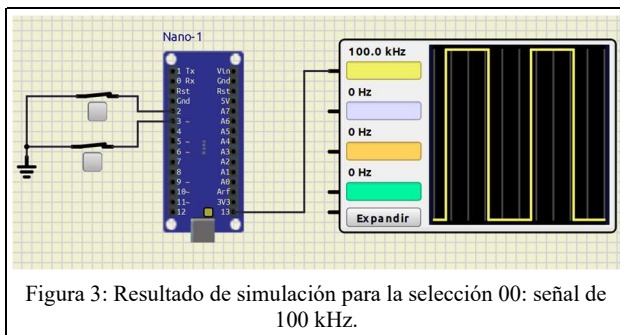


Figura 3: Resultado de simulación para la selección 00: señal de 100 kHz.

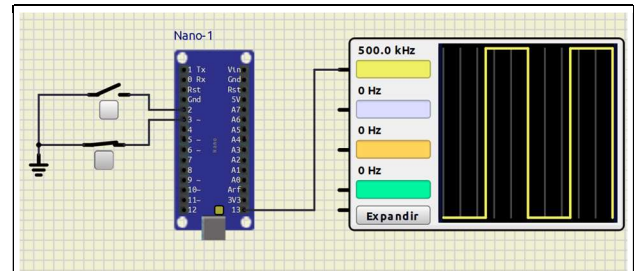


Figura 4: Resultado de simulación para la selección 01: señal de 500 kHz.

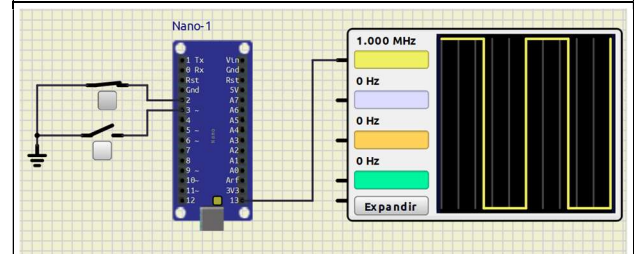


Figura 5: Resultado de simulación para la selección 10: señal de 1 MHz.

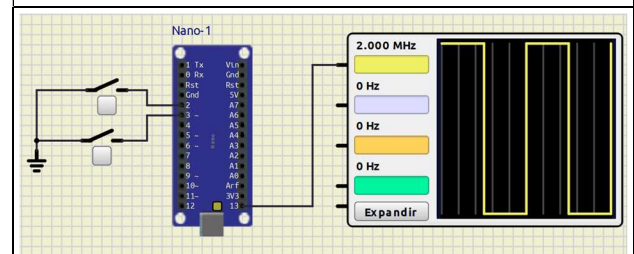


Figura 6: Resultado de simulación para la selección 11: señal de 2 MHz.

En una primera integración del selector se observó que la rutina de 2 MHz quedaba ejecutándose de forma indefinida y no permitía regresar a las demás frecuencias. Esto ocurría porque la secuencia SBI PINB, PINB5 seguida de RJMP FREQ_2M mantenía correctamente los cuatro ciclos necesarios para 2 MHz, pero nunca volvía a leer las entradas. La versión final resolvió este comportamiento mediante SBIS sobre PD2 y PD3, de modo que ambos bits se revisan durante la propia rutina sin alterar la temporización de cuatro ciclos cuando la selección permanece en 11.

PD3:PD2	Frecuencia nominal	Comprobación en SimulIDE
00	100 kHz	Correcta
01	500 kHz	Correcta
10	1 MHz	Correcta
11	2 MHz	Correcta

Con la versión final del programa fue posible cambiar entre las cuatro combinaciones de entrada durante la ejecución. Los pull-up internos permitieron representar un uno lógico con el interruptor abierto y un cero lógico al cerrarlo hacia GND. El frecuencímetro virtual conectado a D13 permitió identificar de forma directa la frecuencia seleccionada.

El archivo hexadecimal generado por Microchip Studio también fue cargado en la tarjeta Arduino Nano física mediante el bootloader serial. La herramienta de programación reportó la escritura y verificación correcta de 114 bytes de memoria Flash. Esto confirma que el firmware generado por el proyecto en

ensamblador quedó almacenado en el ATmega328P. La comprobación física de la forma de onda puede realizarse conectando un osciloscopio o frecuencímetro entre D13 y GND.

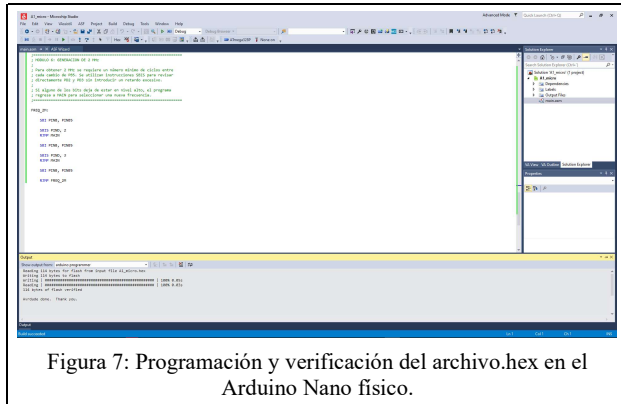


Figura 7: Programación y verificación del archivo.hex en el Arduino Nano físico.

5. Conclusión

El desarrollo de la actividad permitió implementar un generador de frecuencias utilizando directamente los recursos digitales y el conjunto de instrucciones del microcontrolador ATmega328P. Mediante dos entradas binarias fue posible seleccionar cuatro señales de salida diferentes, mientras que PB5 se utilizó para producir una onda cuadrada de 100 kHz, 500 kHz, 1 MHz o 2 MHz. La simulación mostró que la temporización por software depende directamente del número de ciclos consumidos por cada instrucción, por lo que el cálculo y la verificación de dichos ciclos resultan fundamentales para obtener la frecuencia deseada.

También se comprobó el funcionamiento de los registros de dirección, salida y lectura de los puertos AVR, así como el uso de resistencias pull-up internas y de instrucciones de manipulación de bits. La corrección realizada en la rutina de 2 MHz permitió conservar la frecuencia y, al mismo tiempo, detectar cambios en las entradas para regresar al módulo de selección. Finalmente, el archivo hexadecimal fue programado y verificado correctamente en un Arduino Nano, demostrando la relación entre el código ensamblador, la simulación y la implementación sobre hardware real. La actividad permitió comprender de manera práctica cómo la arquitectura, los registros y los tiempos de ejecución del AVR determinan el comportamiento de un sistema embebido.

Referencias

- [1] Microchip Technology Inc., “ATmega328P: 8-bit AVR Microcontroller with 32K Bytes In-System Programmable Flash,” datasheet. Dirección: https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf (visitado 27-09-2026).
- [2] Microchip Technology Inc., “AVR Instruction Set Manual: AVR Instruction Set Summary.” Dirección: <https://onlinedocs.microchip.com/oxy/GUID-74578654-4F5F-4B29-9F3C-6C3B1F1DF670-en-US-2/GUID-64F3D141-2594-4755-9103-550108045801.html> (visitado 27-09-2026).
- [3] Arduino, “Arduino Nano - Technical Specifications,” Arduino Official Store. Dirección: <https://store.arduino.cc/arduino-nano> (visitado 27-09-2026).
- [4] Microchip Technology Inc., “Microchip Studio User Guide,” DS50002718. Dirección: <https://ww1.microchip.com/downloads/aemDocuments/documents/MC>

U08/ProductDocuments/UserGuides/Microchip-Studio-UserGuide-DS50002718.pdf (visitado 27-09-2026).

- [5] SimulIDE, “SimulIDE Circuit Simulator.” Dirección: <https://simulide.com/p/> (visitado 27-09-2026).